> **Referencia en vivo y laboratorio:** para el recorrido sobre el asistente publicado y el laboratorio PostgreSQL/pgvector (`lab/`), consulta también `recorrido-cloud.md` . Este guion articula los 90 minutos de S11 sincronizados con las 15 diapositivas de `material/presentaciones/S11-crag.pptx` .

Guion S11 · Corrective RAG (CRAG): reordenamiento, suficiencia, grafo y métricas

Fecha: martes 27 de octubre de 2026 · **Cohorte:** M/J · **Horario:** 18:30–20:00 `Europe/Madrid` (90 minutos con preguntas).

Presentación: `material/presentaciones/S11-crag.pptx` (`material/presentaciones/S11-crag.pdf`).

Laboratorio del ejercicio: `lab/` (`bun run lab:demo` , `bun run lab:eval` , `lab/eval/resultados-laboratorio.md`).

Demostrador del curso: `bun run dev` (guiones de desarrollo locales) / `https://rag-thepower.josenuevo.dev` .

0–10 min · Objetivo, QR de S10 y un límite real del RAG básico (Diapositivas 1 a 3)

Diapositiva 1 · CRAG: El mismo asistente

Explica:

> En S10 construimos un RAG de una sola búsqueda con umbral previo. Hoy evolucionamos ese mismo proyecto —conservando intacta la etapa de RAG básico como línea base— hacia un grafo de **Corrective RAG (CRAG)** en LangGraph: recuperamos hasta 10 candidatos, separamos el **reordenamiento (`rank`)** de la **valoración de suficiencia (`assess`)**, conservamos o descartamos evidencia y permitimos hasta 2 reformulaciones (3 búsquedas en total) antes de responder o rechazar.

Diapositiva 2 · Antes de empezar (QR)

Deja el QR visible:

> Este QR de AI Engineer evalúa nuestra clase anterior: **S10, la del jueves 22 de octubre**.

Diapositiva 3 · Un límite del RAG básico

Plantea el problema técnico concreto:

- Cuando una pregunta exige conectar dos documentos o dos secciones distintas (por ejemplo, *«¿Cómo modela LangGraph el estado de un agente y cómo se observa esa ejecución?»* entre S5 y S6, o *«¿Cómo se autentican los clientes mTLS y qué tiempo de primera respuesta aplica en un incidente SEV-1?»* en el laboratorio AetherCloud), una única búsqueda `top-4` puede traer solo una de las dos mitades o dejar la segunda bajo el corte.
- Subraya que **CRAG no garantiza ganar en el 100 % de los casos**: añade llamadas, latencia y posibles falsos rechazos si el umbral de suficiencia es estricto. Por eso en S11 es obligatorio medirlo frente al RAG básico.

10–25 min · Arquitectura del grafo: recuperar, reordenar y juzgar suficiencia (Diapositivas 4 a 6)

Diapositiva 4 · RAG básico frente a esta adaptación de CRAG

Compara en la tabla de la diapositiva 4 (inspiración: Yan, Gu, Zhu y Ling, arXiv:2401.15884):

- **Búsquedas:** 1 búsqueda (`top-4`) en RAG básico frente a hasta **3 búsquedas (`top-10`)** y **2 reformulaciones** en CRAG.
- **Control:** un único umbral de similitud en RAG básico frente a dos nodos separados en CRAG: **`rank` (reordenamiento en `[0, 1]`)** y **`assess` (valoración de suficiencia y cobertura)**.
- **Rutas:** responder o rechazar en RAG básico frente a **suficiente (`answer`)**, **parcial/ambigua (`retain → reformulate`)** e **insuficiente (`discard → reformulate`)**.

Diapositiva 5 · Estado del grafo y preservación de la pregunta original

Abre `apps/api/src/crag-graph.ts` (y `lab/src/crag-graph.ts`):

- Señala que `originalQuestion` **nunca se sobrescribe** cuando `currentQuery` cambia tras una reformulación: el evaluador y el generador final comprueban siempre contra lo que preguntó el alumno.
- Cada turno inicializa sus contadores (`retrievalCount: 0` , `reformulationCount: 0`) de cero.

Diapositiva 6 · Diferencia entre recuperar (`retrieve`), reordenar (`rank`) y juzgar (`assess`)

Detente en esta distinción clave del ejercicio (`packages/rag/src/crag.ts` y `lab/src/reranker.ts`):

1. **Recuperar (`retrieve`):** trae hasta **10 candidatos** rápidos desde el índice vectorial (`vector(768)` con HNSW en `lab/` o Vectorize `1024 d` en el curso).
2. **Reordenar (`rank`):** evalúa cada candidato frente a la consulta y le asigna una puntuación en la **escala cerrada `[0, 1]`** (`0,00` = irrelevante; `1,00` = relevancia directa). Enseña también en `lab/src/reranker.ts` la función `formatQwen3RerankerPrompt` con la plantilla oficial `yes` / `no` de `Qwen3-Reranker-0.6B` propuesta como opción en el enunciado.
3. **Juzgar suficiencia (`assess`): relevancia de un fragmento aislado ≠ cobertura de toda la pregunta**. Un fragmento de S5 puede recibir `score = 0,85` (muy relevante para la primera mitad de una pregunta compuesta), pero si falta la segunda mitad sobre S6, la cobertura global de la pregunta es `0,50` y la decisión del nodo `assess` debe ser `partial` , nunca `sufficient` .

25–45 min · Escala `[0, 1]` , umbrales calibrados y recorrido de las rutas (Diapositivas 7 a 9)

Diapositiva 7 · Umbrales calibrados y rutas del grafo

Muestra los umbrales definidos en `CRAG_SCORE_THRESHOLDS` (`packages/rag/src/crag.ts`) y `LAB_CRAG_CONFIG` (`lab/src/config.ts`), calibrados sobre ejemplos de validación distintos del test de 20 preguntas:

- **`minChunkRelevance = 0,50`** : umbral mínimo en `[0, 1]` para que un fragmento entre en `usefulIds` (máximo 6 fragmentos conservados).
- **`partialCoverageMin = 0,40`** : cobertura mínima global cuando existe al menos un fragmento útil (`⩾ 0,50`) pero quedan subpreguntas pendientes en `gaps` → ruta **`partial` (`retain → reformulate → retrieve`)**.

- `sufficientCoverageMin = 0,75`: cobertura global sin lagunas en `gaps` → ruta `sufficient` (`answer`).
- Por debajo de `0,40` de cobertura o sin ningún fragmento `≥ 0,50` → ruta `insufficient` (`discard → reformulate → retrieve`).

Diapositiva 8 · Ejemplo completo de evidencia parcial (`partial-s5-s6` y `lab:demo`)

Recorre paso a paso el caso de evidencia parcial en el navegador (etapa **CRAG**, guion **Evidencia parcial y segunda búsqueda**) y en terminal con `bun run lab:demo`:

1. **Pregunta compuesta:** *«¿Cómo modela LangGraph el estado de un agente y cómo se observa esa ejecución?»*.
2. **Pasada 1:** `retrieve` trae `s05-langgraph:00`; el nodo `rank` le da `0,800` (`≥ 0,50`); el nodo `assess` detecta la laguna (*«Falta cómo observar la ejecución del agente»*, cobertura `0,50 < 0,75`) y emite `partial`.
3. **Conservación y reformulación:** `retain` guarda `s05-langgraph:00`; `reformulate` escribe *«observabilidad Langfuse y LangSmith de un agente»*.
4. **Pasada 2:** `retrieve` recupera `s06-observabilidad:00` (`0,860`); `rank` ordena `S6 (0,860), S5 (0,400)` sin duplicar IDs; `assess` verifica que ambas partes están cubiertas (`sufficient`) y `answer` redacta citando `[cite:c3a59a406888b:s06-observabilidad:00]` y `[cite:c3a59a406888b:s05-langgraph:00]`.

Diapositiva 9 · Descartar y buscar otra vez (`irrelevant-then-s6`)

Selecciona el guion **Primera evidencia descartada**:

- En la búsqueda 1 entra `s16-cloud-aws:00` con puntuación `0,200 < 0,50`.
- `assess` emite `insufficient`; `discard` anota `s16-cloud-aws:00` como descartado en el intento 1; tras reformular, la búsqueda 2 recupera `s06-observabilidad:00` (`0,880`) y la respuesta final **nunca cita ni recibe en su prompt el fragmento descartado S16**.

45–65 min · Límites de turno, fallos controlados y lectura de la traza (Diapositivas 10 y 11)

Diapositiva 10 · Límites de un solo turno

Ejecuta el guion **Dato ausente y rechazo** (beca de 1991) y el caso de criptomonedas en `bun run lab:demo`:

- Muestra que el grafo ejecuta exactamente **3 búsquedas** (la inicial más **2 reformulaciones**) y se detiene en `refuse` con **0 llamadas al generador**.
- Muestra también **Calendario sin cohorte** (`0` búsquedas, `clarify`) y **Fallo controlado del evaluador** (`error`, sin reintentos ocultos, respetando el plazo único de `20 s`).

Diapositiva 11 · La traza que ve la clase

Señala en el panel **Traza de la clase** cómo aparecen desglosados los pasos `retrieve`, `rank` y `assess`, la consulta original frente a la última consulta reformulada, la lista de fragmentos conservados con su puntuación en `[0, 1]` y la lista de descartados con su número de intento.

65–75 min · Lectura de métricas y resultados de comparación (Diapositiva 13)

Pasa a la **Diapositiva 13** y abre `lab/eval/resultados-laboratorio.md` y `material/comparacion/resultados/tabla-resultados.md`:

Lote evaluado (20 preguntas: 14 respondibles + 6 sin respuesta)	Variante	Total	Respondibles (X/14)	Rechazos (Y/6)	Recuperaciones medias	Reformulaciones medias	Mediana latencia
Laboratorio pgvector (`lab/` , 768 d + HNSW)	RAG básico (S10)	**16/20**	10/14	6/6	**1,00**	0,00	1 ms
Laboratorio pgvector (`lab/` , 768 d + HNSW)	CRAG (S11)	**17/20**	11/14	6/6	**1,80**	0,80	4 ms
Histórico cloud 03/10/2026 (BGE-M3 1024 d)	RAG básico	**17/20**	11/14	6/6	**0,95**	0,00	1193 ms
Histórico cloud 03/10/2026 (BGE-M3 1024 d)	CRAG	**18/20**	13/14	5/6	**1,55**	0,60	4304 ms

Explica las tres conclusiones docentes:

1. **Por qué mejora en preguntas compuestas:** en el laboratorio (`lab/`), el caso 14 (mTLS + incidente SEV-1) y los casos 5 y 10 fallan en RAG básico porque `top-4` pierde una sección, mientras que CRAG los resuelve con `top-10` , `rerank` y `assess` .
2. **El coste de iterar (recuperaciones medias):** CRAG sube de `1,00` a `1,80` búsquedas medias en el laboratorio (y de `0,95` a `1,55` en el lote histórico cloud, multiplicando por `3,6x` la mediana de latencia de `1,19 s` a `4,30 s`) porque cada pregunta sin respuesta agota las 3 búsquedas antes de rechazar.
3. **Honestidad sobre la referencia:** explica que el lote cloud del 3 de octubre es **histórico** (ejecutado con la versión congelada de aquel día antes de pasar a `claude-haiku-5-5` y separar `rank` y `assess`), y que para la referencia actual se ha dejado congelado un segundo conjunto nuevo en `material/comparacion/conjunto-reservado-referencia-actual-20.md` .

75–80 min · Enunciado publicado de S11 y criterios de entrega (Diapositiva 12)

Abre `material/ejercicios/S11-enunciado.md` :

- Proyecto **reconstruible desde 8 o más documentos ficticios e índice vacío** en PostgreSQL/pgvector (`vector(768)` + HNSW), **conservando el RAG básico de S10**.

- Grafo en **LangGraph** con pasos separados: `retrieve` (hasta 10 candidatos) → `rerank` (escala explicada `[0, 1]`, con `Qwen3-Reranker-0.6B` como opción) → `assess` (suficiencia con umbrales calibrados, separando relevancia de un fragmento y cobertura total) → `retain` / `discard` → `reformulate` (máximo 2 reformulaciones, 3 búsquedas en total) → `answer` / `refuse`.
- **Comparación ejecutada sobre 20 preguntas nuevas (14 respondibles y 6 no respondibles)** informando `X/14`, `Y/6`, **recuperaciones medias**, latencia y revisión manual de fuentes.

80–90 min · Preguntas y cierre (Diapositivas 14 y 15)

- Deja visible la **Diapositiva 15** con el QR de evaluación de la clase anterior (S10, 22 de octubre).